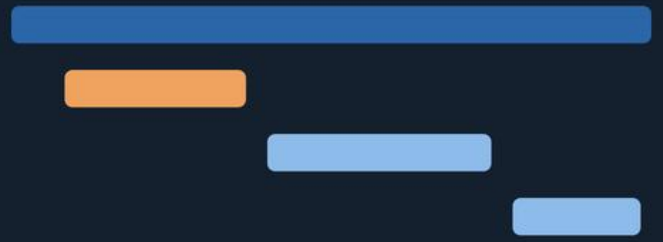
CONF42 PROMPT ENGINEERING 2026

You Cannot Fix a Prompt You Cannot See Fail

Debugging the plan, not just the answer



Krishna Chaitanya Balusu

Senior Software Engineer, Meta

ABOUT ME

A little about me

Senior Software Engineer at Meta. I build AI agents for incident response and work on how we observe them.

Contributed the **plan** operation to the OpenTelemetry GenAI semantic conventions (PR #97, merged May 2026).

Built AgentTelemetry, a benchmark for finding faults in agent traces. Published at ACM AIware 2026.

ROADMAP

What we'll cover

01

The problem

Why agent prompts are hard to debug

02

Make the plan visible

The plan span in OpenTelemetry

03

Demo

Screenshots from a real run

04

Better prompts

From a trace to a prompt change

THE PROBLEM

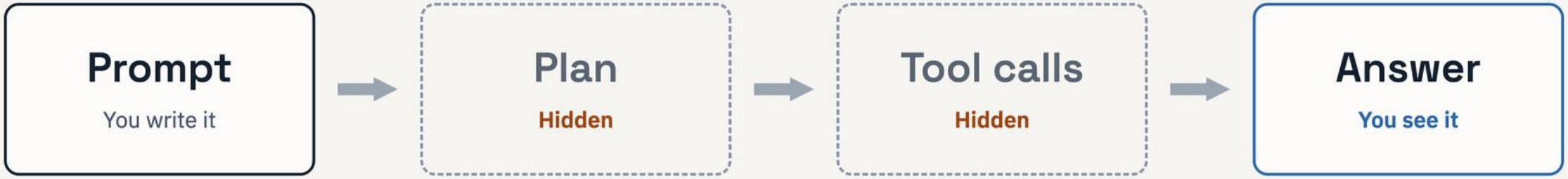
Single prompts have a fast loop



Then repeat. Ten rounds take a few minutes, because the output is the whole story.

THE PROBLEM

Agents hide the step that went wrong



You are tuning a plan you can’t see.

A correct answer can come from a wrong plan.

Your answer check passes. Next time, the same plan gives a wrong answer.

A SMALL EXAMPLE

Meet the release helper

“Is this release ready to ship?”

THE RULES

READY when every required check passes

BLOCKED when a required check fails

UNKNOWN when it can’t get the evidence

THREE TOOLS

`read_summary`

Yesterday’s summary. Fast, but old.

`read_manifest`

Today’s list of required checks.

`read_checks`

Today’s results for those checks.

A SMALL EXAMPLE

Two plans, same answer

PLAN A

```
1. read_summary
answer: READY
```

Answer check: **PASS**

Read today's evidence: **No**

PLAN B

```
1. read_manifest
2. read_checks
answer: READY
```

Answer check: **PASS**

Read today's evidence: **Yes**

Tomorrow a check fails. Plan A still says READY.

MAKE THE PLAN VISIBLE

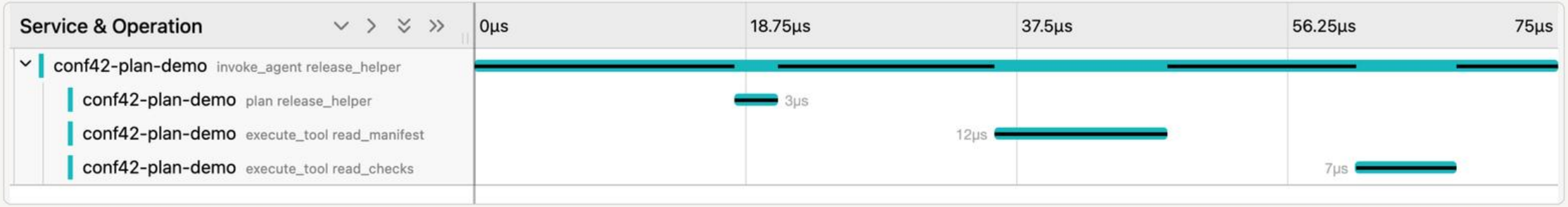
Traces and spans in one minute

Trace

One run of your agent, from question to answer.

Span

One step in that run. It has a name, a start, an end, and attributes: key value facts about the step.



A real trace from the demo, in Jaeger. Top row: the whole run. Below it: one plan span and two tool spans.

MAKE THE PLAN VISIBLE

OpenTelemetry now has a plan span

```
plan release_helper
gen_ai.operation.name = "plan"
gen_ai.agent.name = "release_helper"
demo.plan.tools = [
  "read_manifest",
  "read_checks"
]
```

A shared name

Part of the GenAI semantic conventions. Added in PR #97.

Your plan, your field

demo.plan.tools is my own attribute for this demo.

Still in Development

Pin the version you use.

It records the plan your app got back from the model. It does not record the model’s private thoughts.

MAKE THE PLAN VISIBLE

What counts as a plan?

A planner call

You ask the model for a list of steps, then run them.

A tool choice step

The model picks its tools before it calls any.

A routing step

The agent picks which sub agent handles the task.

Add a plan span when your app has a clear planning step. If planning and acting happen in one model call, start with tool spans.

MAKE THE PLAN VISIBLE

Recording the plan takes a few lines

```
with tracer.start_as_current_span("plan release_helper") as span:
    span.set_attribute("gen_ai.operation.name", "plan")
    span.set_attribute("gen_ai.agent.name", "release_helper")
    plan = make_plan(prompt, task)  # your model call
    span.set_attribute("demo.plan.tools", plan.tools)

for tool in plan.tools:
    with tracer.start_as_current_span(f"execute_tool {tool}") as s:
        s.set_attribute("gen_ai.operation.name", "execute_tool")
        s.set_attribute("gen_ai.tool.name", tool)
        run_tool(tool)
```

make_plan and run_tool stand in for your own code. The full demo is a short Python file.

MAKE THE PLAN VISIBLE

Then check the plan like a test

```
tools = plan_span.attributes["demo.plan.tools"]
has_both = "read_manifest" in tools and "read_checks" in tools
in_order = has_both and tools.index("read_manifest") < tools.index("read_checks")
plan_check = "PASS" if in_order else "FAIL"
# no plan span at all: plan_check = "UNKNOWN"
```

PASS

The plan reads the manifest, then the checks.

FAIL

The plan skips one, or reads them out of order.

UNKNOWN

There is no plan span to check.

MAKE THE PLAN VISIBLE

Why one shared name matters

Every framework logs planning its own way, if it logs it at all. One shared name means one query finds every plan, across frameworks, prompt versions and models.

6 of 14

fault types caught by standard OpenTelemetry and the GenAI conventions, before the plan operation existed. Planning failure was one of the 8 they missed.

14 of 14

caught with agent specific spans, including a planning span.

Source: AgentTelemetry benchmark, ACM AIware 2026. Controlled fault injection.

DEMO

The demo: five runs, four checks

- Recorded ahead of time. Every image is a screenshot of a real run.
- Scripted plans, so every run is repeatable.
- Real OpenTelemetry spans, sent to Jaeger.
- No model calls. Runs on a laptop with no API key.

Answer check
Did it give the expected answer?

Plan check
Manifest, then checks, in that order?

Tools
Did every tool call succeed?

Evidence
Did it read today's evidence?

DEMO

One command, five runs

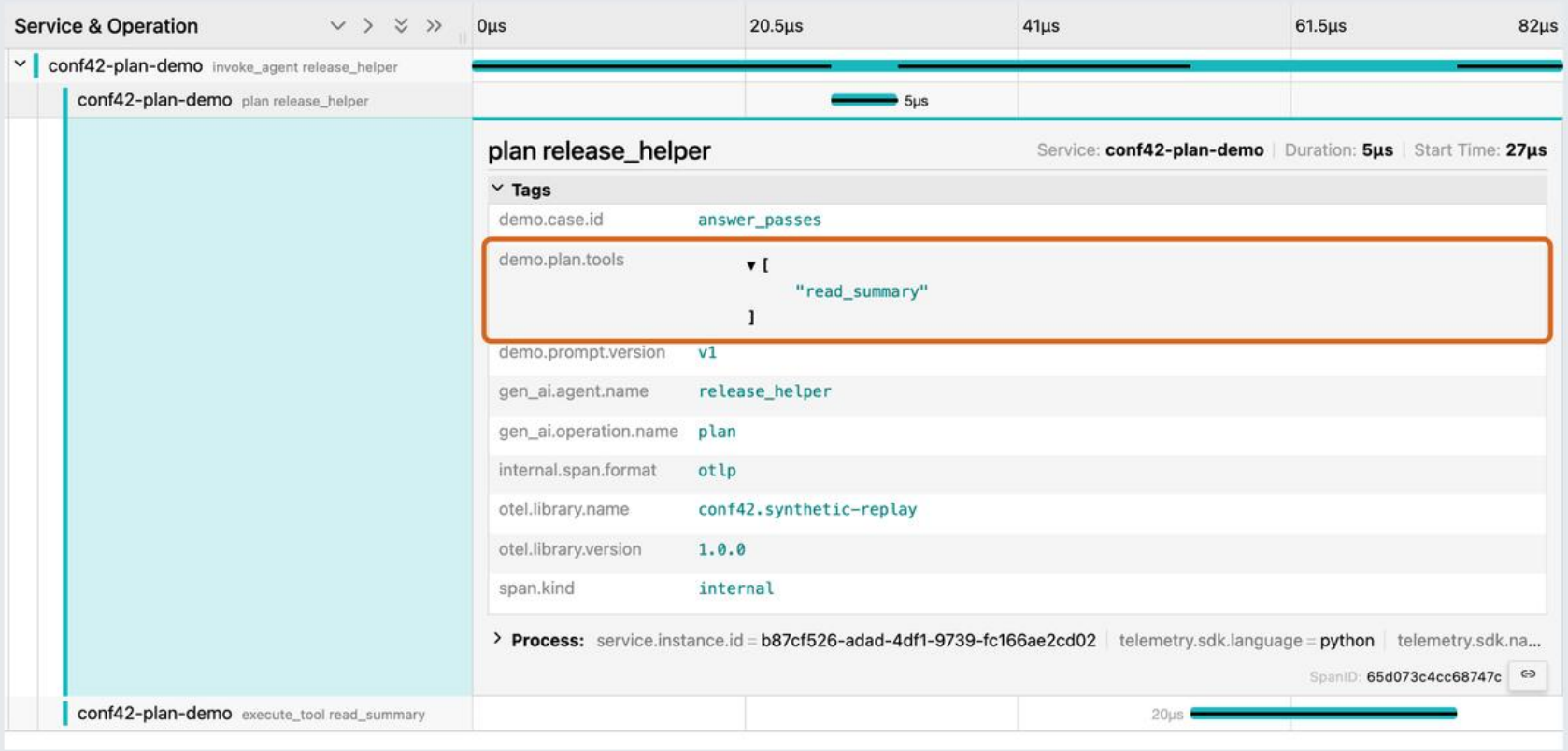
conf42-prompt-engineering-2026

```
(demo-venv) $ python demo/replay.py --out .build/rehearsal-results.json
SYNTHETIC REPLAY: scripted plans, real OTel spans, zero model calls
case              answer    answer check  plan check   tools      evidence
answer_passes     READY     PASS          FAIL         SUCCESS    FAIL
revised_plan      READY     PASS          PASS         SUCCESS    PASS
tool_timeout      UNKNOWN   UNKNOWN       PASS         ERROR      FAIL
missing_plan      READY     PASS          UNKNOWN      SUCCESS    PASS
stale_summary     READY     FAIL          FAIL         SUCCESS    FAIL

Span export: .build/rehearsal-results.json
```

DEMO: ANSWER_PASSES

The answer passed. The plan did not.



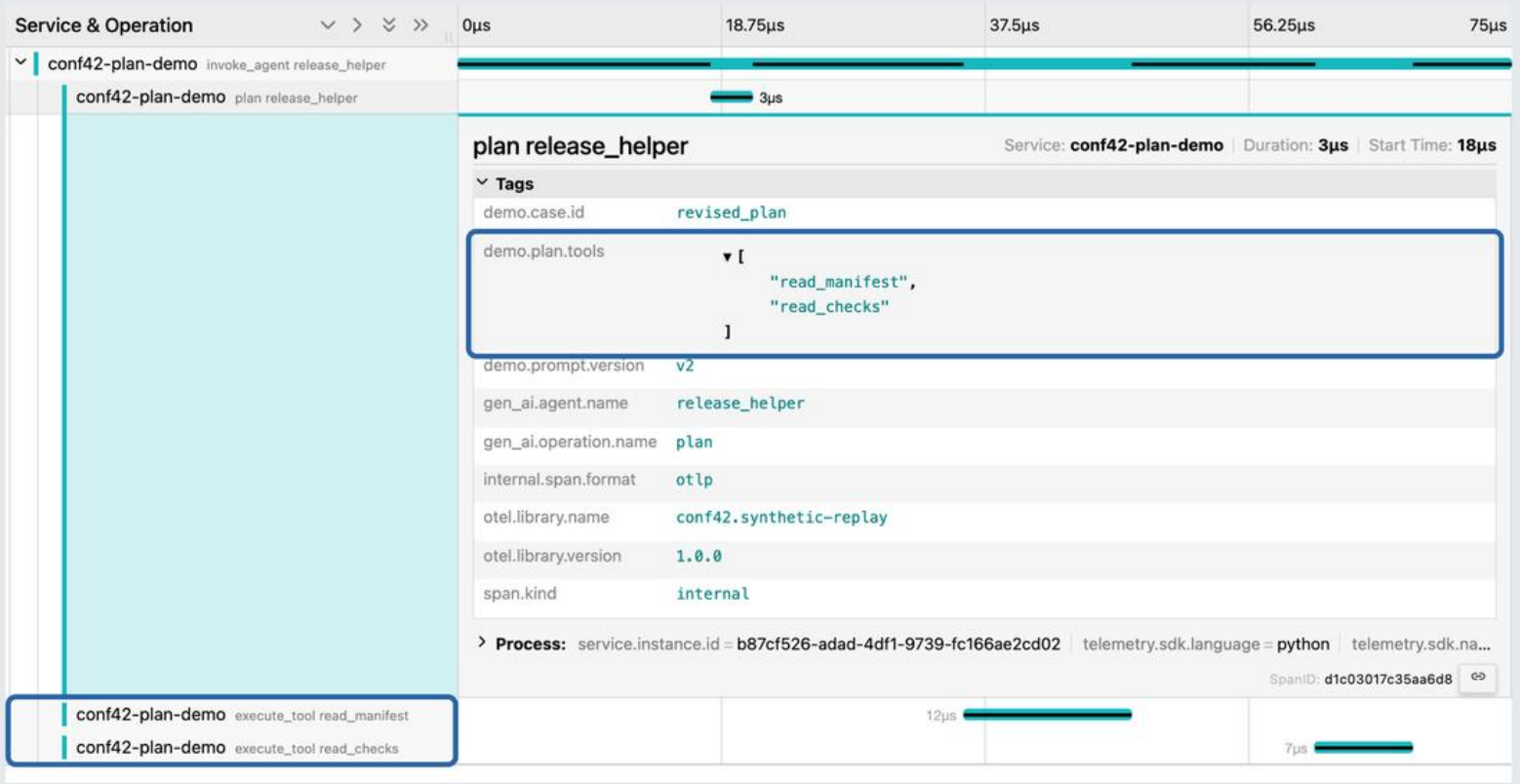
`demo.plan.tools`

`read_summary`

No manifest. No checks. The answer was still right.

DEMO: REVISED_PLAN

The revised plan does the work



`demo.plan.tools`

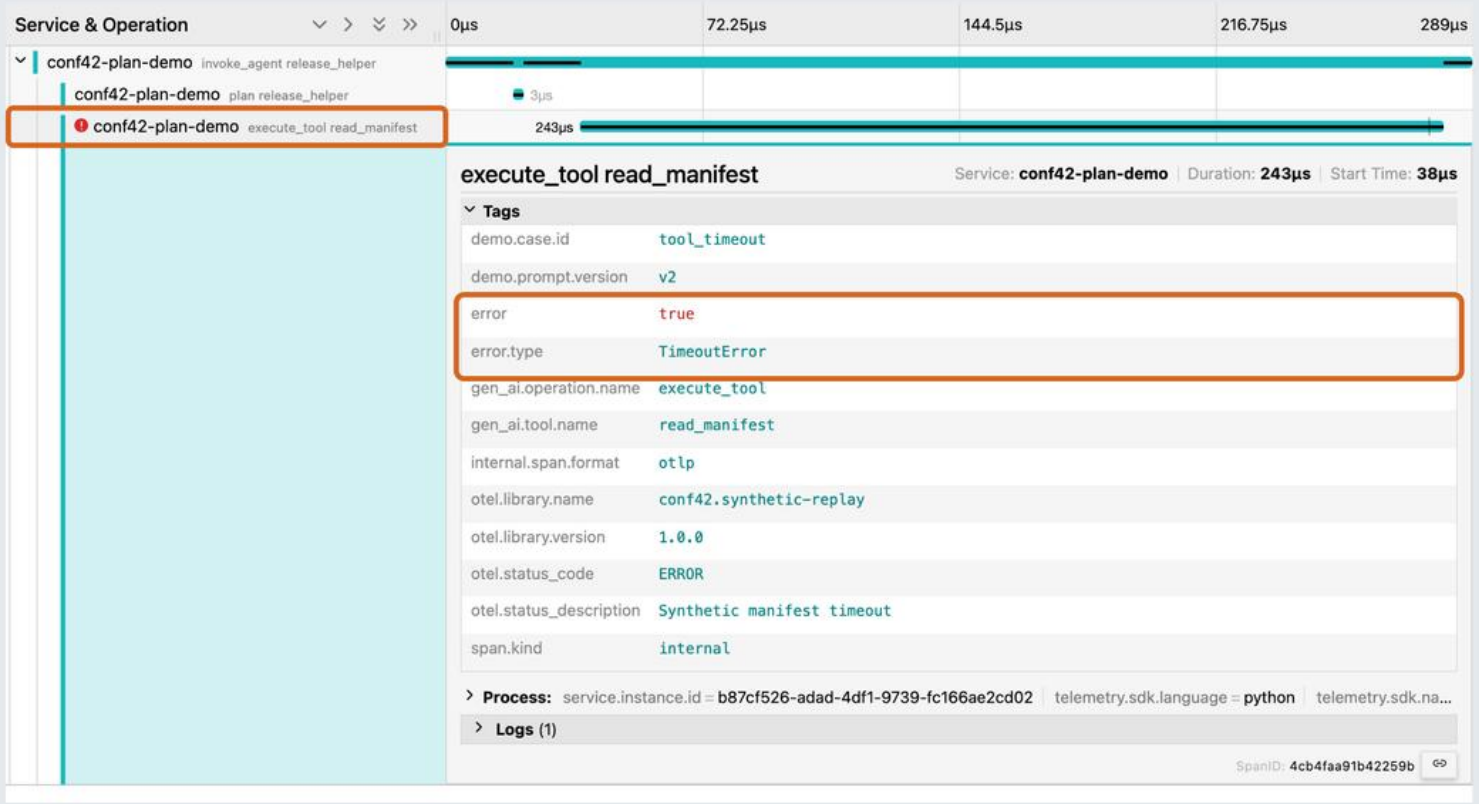
`read_manifest`

`read_checks`

Two tool spans follow, in the same order.

DEMO: TOOL_TIMEOUT

Good plan, broken tool



Plan check: PASS

Tools: ERROR, TimeoutError

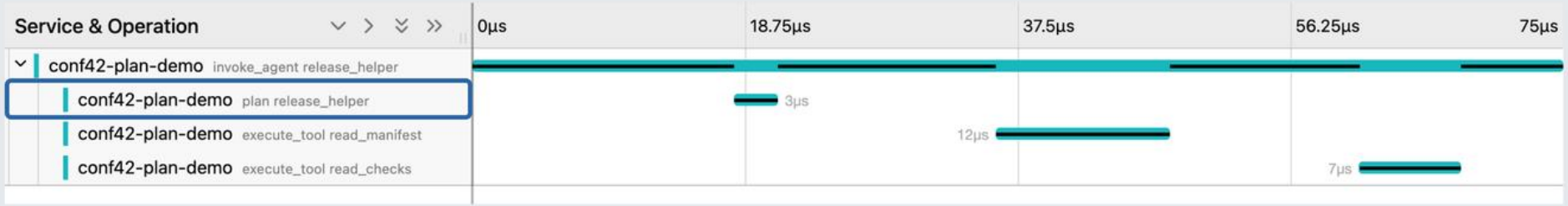
Answer: UNKNOWN

Fix the tool, its timeout, or its retry.

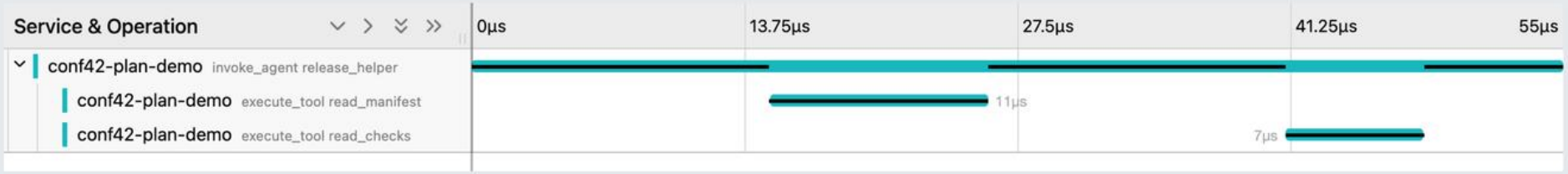
DEMO: MISSING_PLAN

No plan span, no plan verdict

WITH A PLAN SPAN



PLAN SPAN MISSING



Answer check: **PASS**. Plan check: **UNKNOWN**. Fix the instrumentation before you judge the prompt.

DEMO: STALE_SUMMARY

The day the shortcut bites

Today	case	answer	answer check	plan check	evidence
Checks pass	answer_passes	READY	PASS	FAIL	FAIL
Checks fail	stale_summary	READY	FAIL	FAIL	FAIL

Plan check

Failed on both days. It caught the risk early.

Answer check

Failed only on the bad day. It caught the risk late.

Rows copied from the run on slide 16.

BETTER PROMPTS

Which fix goes where

What the trace shows	What to fix
The plan is wrong	The prompt, and the context the agent had when it planned
Good plan, a tool failed	The tool, its timeout, or its retry
Good plan, tools did something else	The code that runs the plan
No plan span	Your instrumentation, first

The trace tells you which row you’re in.

BETTER PROMPTS

The prompt change the trace points to

VERSION 1

"Check release readiness and return READY or BLOCKED."

VERSION 2

"**Read the current release manifest. Verify every check it requires.** Return READY only when all required checks pass. **Return UNKNOWN when required evidence is unavailable.**"

Names the source

Asks for every check

Says what to do when evidence is missing

Prompts copied from demo/replay.py

BETTER PROMPTS

Test the change like an experiment

01

Keep the model, tools and test cases the same.

02

Run each prompt many times. One good run proves little.

03

Score the answer and the plan separately.

04

Keep every failure as a test case.

In this demo the version 2 plan is scripted. With a real model, this is where you measure how often the plan comes out right.

TRY IT

Start this week

- Pick one agent with a clear planning step
- Write down the evidence the task needs
- Wrap the planning step in a plan span
- Add one plan check next to your answer check
- Keep every failure as a test

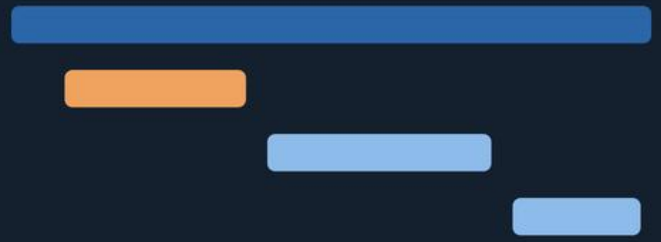
One plan span and one check already give you a better loop.

Check the plan, not just the answer.

Thanks for watching.

linkedin.com/in/kcbalusu

Questions: the Conf42 Discord



REFERENCES

Sources and tools

OpenTelemetry GenAI semantic conventions: agent spans, plan operation

github.com/open-telemetry/semantic-conventions-genai, docs/gen-ai/gen-ai-agent-spans.md. Status: Development.

Plan operation for agent planning spans, PR #97

github.com/open-telemetry/semantic-conventions-genai/pull/97. Merged May 11, 2026.

AgentTelemetry: A Fault Detection Benchmark and Toolkit for LLM Agent Observability

Krishna Chaitanya Balusu. ACM AIware 2026.

Demo setup

OpenTelemetry Python SDK 1.45.1 and Jaeger 1.60. Scripted plans, real spans, zero model calls.